

NILAY360

Complete Master Handoff, Technical Reference & Native App Product Requirements Document

India's Premium Real Estate Platform — Hyderabad

Prepared for: Vanith Kandre, Founder & Product Architect

Purpose: Full-context handoff for continued development across Claude sessions, Claude Code, and Replit

Contains: Session recap, complete technical blueprint, product requirements, native app PRD/TRD, action steps

Generated: 24 September 2026

Table of Contents

How To Use This Document

-
- 1. Project Snapshot
 - 2. Technical Stack (Web Platform)
 - 3. Exact Git State
 - 4. Completed This Session (Verified, Detailed)
 - 5. Active Blockers — Require Direct Action
 - 6. Google Maps — Complete Phase-by-Phase Status
 - 7. Database, Feature & Admin Reference
 - 8. Complete Reminder List
 - 9. Existing Web App Screen Inventory
 - 10. Native Mobile App — Product & Technical Requirements
 - 11. Master Prompts — Ready to Send
 - 12. Final Steps Checklist
-

How To Use This Document

This is a single, complete source of truth for NILAY360's current state and next steps. It is built so that a new AI session, a new developer, or Replit's own build agent can pick up development with zero prior context and zero guesswork.

Steps you need to take

1. Read Section 1 (Project Snapshot) first — it orients you to what NILAY360 is and where it stands today.
2. If you are starting a NEW Claude conversation (web app work): paste this entire document as your first message, then send the prompt in Section 9 (Master Prompt — Web/Claude Code Continuation).
3. If you are starting the native app build in Replit: paste this entire document into Replit's agent as project context, then send the prompt in Section 10 (Master Prompt — Native App / Replit Build).
4. Before either session touches git, have it confirm current repository state matches Section 3 (Exact Git State) — do not let it assume.
5. Complete the 4 blockers in Section 5 yourself (Google Cloud Console, MSG91 dashboard) — these require your direct account access and cannot be done by any AI session.
6. Keep this document updated after each major session — regenerate it rather than relying on any single chat's memory.

What this document is NOT

- It is not a substitute for reading the actual codebase — every AI session should still verify live state, not just trust this document blindly.
- It does not contain secret values (API keys, passwords) — only variable names and what they're for.
- It is a snapshot as of the date below — always note when it was generated and treat anything after that date as unknown until re-verified.

1. Project Snapshot

1.1 What NILAY360 is

NILAY360 is a mass-market PropTech platform for the Indian real estate market, launching from Hyderabad. Site metadata describes it as "a technology-powered real estate platform offering property discovery, virtual tours, and end-to-end transaction support across India." The platform serves four core user types: buyers/renters, owners/sellers, agents, and admins/moderators.

A sister luxury brand, Nivila (NIVILA), targets HNI buyers, NRI investors, and luxury developers as a separate, more premium-positioned product.

1.2 Key identifiers

Live domain	nilay360.com
Instagram	@nilay360_ (instagram.com/nilay360_)
Business phone (current, live)	+91 70933 36360 — replaced old number 7075792497 this session
Founder / Product Architect	Vanith Kandre
Supabase project ref	xjdarhbzrpybqyqeeshy
Hosting	Vercel, production branch: main
Production commit (as of this document)	c41c6d3

1.3 Who's building what

- Website (Next.js, this document's primary subject) — built and maintained by Vanith directly, via Claude Code prompts reviewed in Claude chat sessions.
- Existing mobile wrapper — a Capacitor-based app wrapping the website, built by Vanith's brother. PR #26 added Android/iOS platform setup. This wrapper is being SUPERSEDED, not extended further — see Section 8.
- New native mobile app (decision made this session) — to be built fresh in Replit, NOT Capacitor, sharing the same Supabase backend/database as the website. See Section 10 for full PRD/TRD.
- Proprietary 360° virtual tour platform — a separate, longer-horizon initiative (14-month, six-phase roadmap: WebGL viewer, upload pipeline, dashboard, WebXR VR mode, 3D interior design layer) exploring a full-stack Kuula/Matterport alternative. Not covered in build detail here — flagged for context only.

2. Technical Stack (Web Platform)

2.1 Core framework

- Next.js 16.2.9 (App Router, Turbopack), React 19.2.4, TypeScript ^5
- Note: the root README.md incorrectly claims Next.js 14 — README is stale and should not be trusted; this is a known reminder item.

2.2 Database / backend

- Supabase — @supabase/ssr, @supabase/supabase-js. Postgres + Auth + Storage.
- typescript.ignoreBuildErrors: true in next.config.ts — production builds do NOT fail on TypeScript errors. This is why pre-existing type errors elsewhere don't block deploys.

2.3 Hosting & infrastructure

- Vercel — one cron job defined: GET /api/cron/check-follow-ups at 0 3 * * * (daily 3am UTC). Confirmed via real production logs to be executing successfully.

2.4 UI / Styling

- Tailwind CSS 4, Radix UI primitives, class-variance-authority, clsx, tailwind-merge, lucide-react, next-themes, sonner (toasts)
- Design system colors: Deep Navy (#07111F), Primary Blue (#0B1F3A), Champagne Gold (#C9A227); typography: Manrope + Inter
- A teal / Cal Sans rebrand has also landed on parts of the homepage — whether the resulting palette differentiation across pages is intentional was flagged but never confirmed.

2.5 Forms, animation, media

- Forms: react-hook-form, @hookform/resolvers, zod
- Animation: framer-motion, gsap, lenis (smooth scroll) — scroll mechanics reverse-engineered from lutstudios.com (Next.js + Framer Motion, no scroll-hijacking libraries)
- Media: cloundinary, next-cloundinary, tus-js-client (resumable uploads, used for Bunny Stream video)
- Other: date-fns, recharts (agent analytics), resend (email), posthog-js, @sentry/nextjs

2.6 Third-party integrations — confirmed real, with live API calls

Bunny Stream (video)	Real. Library ID 755427. src/lib/bunny.ts, /api/upload-video, /api/bunny-webhook
WhatsApp via MSG91	Real send capability (OTP working); automated WhatsApp notifications currently non-functional in production — see Blocker C, Section 5
Google Maps / Places / Geocoding	Real. See Section 6 for full phase-by-phase status
PostHog	Real, initialized via PostHogProvider
Cloundinary	Real — image/document uploads

Bunny Stream (video)	Real. Library ID 755427. src/lib/bunny.ts, /api/upload-video, /api/bunny-webhook
Sentry	Real — error tracking, all three configs (client/server/edge) initialized

3. Exact Git State (as of document generation)

Local HEAD = origin/main = commit c41c6d3. In sync — nothing lost, nothing accidentally pushed. STANDING RULE, carried into any new session: nothing is committed or pushed to git without Vanith's explicit, direct go-ahead, every single time, no exceptions.

3.1 Modified, uncommitted files (19)

- connect/page.tsx, contact/page.tsx, pricing/page.tsx, privacy/page.tsx — contact-number system wiring
- FloatingContactMenu.tsx, Footer.tsx, SiteChrome.tsx, constants/index.ts — contact-number system wiring (SiteChrome.tsx is the REAL live footer; Footer.tsx is confirmed dead code, unused)
- properties/page.tsx — Google Maps Phases 4, 5, 6 (multi-property map view, radius search, distance badges)
- PropertyDetailClient.tsx — Google Maps Phases 6, 8 (distance badges, Street View toggle)
- LocationPicker.tsx — Places Autocomplete API migration (Phase 3)
- liveStats.ts — resilience fix (loading/success/error states, auto-retry, never shows fake zero)
- admin/page.tsx — Contact Numbers admin section, plus earlier session's Listings consolidation
- compare/page.tsx, locations/page.tsx — fixed to query live property_listings instead of dead legacy catalog tables
- forgot-password/page.tsx — real Supabase password reset email now sends
- page.tsx (homepage), ThankYouClient.tsx, ScrollHero.tsx — resilience fix + contact-number wiring

3.2 Untracked, new files (6)

- src/hooks/useSiteContact.ts — singular + plural contact-fetching hooks
- src/lib/contactFormat.ts — shared digitsOnly/telHref/waHref/smsHref helpers
- supabase/migrations/075_site_contacts.sql — contact-number table migration (APPLIED live in Supabase)
- scripts/check-streetview-permission.mjs — standalone Node script to re-test Street View API key permission without needing an AI session
- NILAY360_APP_DESIGN_BRIEF.md — full ~60-screen native app design brief (condensed in Section 9 of this document)
- NILAY360_BLUEPRINT.md — full technical blueprint document (condensed across Sections 2, 4, 6 and 7 of this document)

4. Completed This Session (Verified, Detailed)

4.1 Contact Number System — full rebuild

New table `site_contacts` (`contact_type` as PRIMARY KEY: `general` / `customer_care` / `sales` / `capture_team`, plus `label`, `phone`, `whatsapp`, `is_active`, `updated_at`, `updated_by`). RLS: public reads active rows only; admin reads/writes everything. Migration 075 is APPLIED live, confirmed via direct SELECT.

- general seeded with the new number +91 70933 36360
- New admin panel section "Contact Numbers" — add/edit/hide per contact type, every write logged to `admin_audit_log` same as every other admin action
- Shared helpers: `src/lib/contactFormat.ts` (one stored value, never duplicated — derives `tel:/wa.me/sms:` links)
- Shared hooks: `useSiteContact(type)` singular + `useSiteContacts()` plural, mirrors the existing `useLiveStats` hook pattern
- 7 display surfaces rewired: Contact page (full multi-department loop), Connect page footer column (full loop), Privacy page (full loop), Pricing page (swapped to 'sales' type with 'general' fallback — smarter than a blanket expansion), `FloatingContactMenu` (WhatsApp pill became an expandable accordion of departments; SMS stayed general-only), `ThankYouClient/ScrollHero` (deliberately kept general-only — no decision friction right after a conversion), `SiteChrome`'s real live footer (single line, space-constrained)
- `BRAND.phone` / `BRAND.whatsapp` removed entirely from `constants/index.ts` — zero lingering hardcoded fallback anywhere in the codebase
- Confirmed and deliberately untouched: `MSG91_WHATSAPP_INTEGRATED_NUMBER` (the number automated WhatsApp messages send FROM) is a separate infrastructure value from what's displayed to visitors — changing `site_contacts` does not and should not change this

4.2 Live Stats Resilience Fix

`liveStats.ts` fully rewritten with an explicit status field: `loading` / `success` / `error`. The function now throws on any real failure instead of silently returning zeros. `useLiveStats()` automatically retries twice (0.6s, then 1.8s delay) before giving up. Homepage hero tiles now render "—" instead of a fake "0" on failure; trust-bar copy omits number-dependent fragments until real data lands.

- The identical anti-pattern was independently found and fixed on `locations/page.tsx`'s own separate count-fetching effect
- 5 other pages (blog, commercial, careers, rent, buy) consume `useLiveStats()` and automatically inherit the safer data layer, but were not individually rewritten for UI-level status gating in their prose — a deliberate, stated scope boundary, not a silent gap

- **IMPORTANT CONTEXT:** the repeated "0 Listings / 0 Agents / 0 Cities" scares tonight were confirmed, TWICE, independently, to be issues local to Vanith's own machine — first a browser-profile/extension conflict (resolved via Incognito), second a total local network failure affecting Supabase AND PostHog simultaneously (also resolved via Incognito). The live production site was confirmed healthy throughout both incidents via clean Vercel logs (zero 500 errors) and the correct commit being deployed. This resilience fix is still genuinely valuable — it protects against real future transient failures, not just tonight's false alarms.

4.3 Google Maps — see Section 6 for the full phase table

Summary: Phases 1, 2, 3, 4, 5, 6, 7 are DONE (several were found already built through investigation, not newly written). Phase 8 (Street View) is code-complete but blocked on a Google Cloud Console API key issue. Phase 9 (natural-language search) is deliberately deferred to its own future session.

4.4 Forgot Password

Real Supabase `resetPasswordForEmail()` call now wired in — this was previously a complete no-op that silently redirected to Login with no email ever sent. The success message is deliberately non-committal (does not reveal whether an email is registered), matching Supabase's own security convention.

KNOWN, DELIBERATELY DEFERRED GAP: there is still no "set new password" screen anywhere in the app. The reset email's link lands a user back on the site, signed in, with no UI to actually change their password. Vanith explicitly said to deprioritize this for now — it remains open, not forgotten.

4.5 Data Integrity Fixes

- `/locations` — now queries live `property_listings` instead of the dead legacy `properties/cities` catalog tables
- `/compare` — same fix, with explicit field-name remapping (`property_category` -> `property_type`, `locality` -> `neighbourhood`, `built_up_area` -> `area_sqft`, `photo_urls` -> `images`); `parking_spaces/year_built` deliberately left null since those columns don't exist on the real table, matching how `PropertyDetailClient.tsx` already handles that same gap elsewhere

4.6 Assign Agent Dropdown — Diagnosed Only, NOT Rebuilt

Confirmed real bug: the dropdown's underlying query has no artificial limit and correctly fetches all approved agents, but the `useEffect` only runs once per page load with zero refetch trigger — so newly-approved agents (confirmed missing in testing: NSR REALTY SERVICES, Manikanta) don't appear until a hard page refresh.

Vanith asked for an ADVANCED rebuild: real search/filter, pagination or virtual scrolling for hundreds/thousands of agents, and newest/oldest sort. This was diagnosed only — the advanced rebuild itself was never started. This is a real, concrete next task.

5. Active Blockers — Require Vanith's Direct Action, Not More Code

These four items cannot be resolved by any AI coding session. Each requires Vanith to act directly in an external dashboard (Google Cloud Console or MSG91).

Blocker A — Street View API key (Google Maps Phase 8)

The code is complete and verified correct through extensive live debugging (isolated server-side vs. browser-side calls, tested against known-good landmark coordinates like Charminar, tested with multiple parameter combinations). The pattern strongly suggests the Google Cloud Console API key is currently in a mid-edit or inconsistent state — coordinates that returned real results earlier in the session later failed even via the exact same server-side path that had worked.

1. Go to Google Cloud Console -> APIs & Services -> Credentials
2. Click the key used for NEXT_PUBLIC_GOOGLE_MAPS_API_KEY
3. Under "API restrictions," confirm "Street View Static API" is checked in the allowed list
4. Click SAVE at the bottom of the page (a common miss is editing the list without saving)
5. Wait 5-10 minutes for propagation
6. Run: node scripts/check-streetview-permission.mjs (already built, needs no AI session to use)

Blocker B — Google Maps Marker deprecation (needs a Map ID)

google.maps.Marker is deprecated in favor of AdvancedMarkerElement across 3 usage sites (NearbyPlacesMap, PropertyLocationMap, LocationPicker). AdvancedMarkerElement REQUIRES a mapId on the Map instance — but once a map has a mapId, Google ignores the inline JS styles array entirely. NILAY360's entire dark navy/teal map theme is built from that inline styles array, so this migration cannot be a pure code change without a matching Cloud-Console-configured style profile.

1. In Google Cloud Console, go to Map Management and create a new Map ID
2. Build a cloud-based style profile replicating the existing DARK_MAP_STYLES array (a Claude Code session can help translate the existing JSON into Cloud Console's style editor format once you're in there)
3. Hand the new Map ID to a Claude Code session — the actual marker code swap is mechanical once the ID exists

Note: this is not urgent. google.maps.Marker still works today; Google gives roughly 12 months notice before actual removal.

Blocker C — WhatsApp Business automated notifications (MSG91)

Freshly re-verified this session:

- Code IS committed and pushed (whatsapp.ts + all 6 notify-* routes, commit 7c97e1b, live in the current production build)

- "TEMPORARY" debug console.log statements are still present at whatsapp.ts lines 71 and 91 — meant to be removed only after one confirmed real delivery, which has never happened
 - MSG91_WHATSAPP_INTEGRATED_NUMBER and MSG91_WHATSAPP_TEMPLATE_NAME DO NOT EXIST in Vercel's production environment variables — reconfirmed absent this session. Automated WhatsApp sending is completely non-functional in production right now, despite the code being fully deployed.
 - The Meta-submitted message template "nilay360_notification" was last known status "pending" approval — never confirmed approved. This cannot be checked from code.
1. Log into the MSG91 dashboard and check the real, current approval status of the nilay360_notification template
 2. If approved: retrieve the real WhatsApp Business sending number and confirm the exact template name
 3. Add MSG91_WHATSAPP_INTEGRATED_NUMBER and MSG91_WHATSAPP_TEMPLATE_NAME to Vercel's production environment variables
 4. Only then have a Claude Code session test one real end-to-end delivery, and only after that succeeds, remove the temporary debug logging

Blocker D — Router / ISP DNS-TLS interference

Confirmed real, affecting Vanith's entire home network (both LAN cable and WiFi), on top of the separate browser-profile issue also found this session. Evidence: raw curl TLS handshake failures, and a suspicious DNS collision where two completely unrelated Supabase domains resolved to the identical IP address — a strong signature of ISP-level or router-level DNS interception, not a genuine Supabase-side problem.

Not a code problem. Not urgent tonight, but has caused real confusion and wasted debugging time on at least two separate occasions. Worth a call to the ISP, or manually setting the router's DNS to 8.8.8.8 / 1.1.1.1, whenever convenient.

6. Google Maps — Complete Phase-by-Phase Status

This is the real, originally-approved 9-phase plan, recovered from an earlier Claude conversation (not invented this session). The vision: turn NILAY360 into a location-intelligent platform using the Maps JavaScript API, Places API/Autocomplete, Geocoding API, Routes API, Street View, and eventually natural-language smart search.

#	Phase	Status
1	Foundation (lat/lng schema, geocoding on create/edit, backfill)	DONE (pre-existing, migration 060)
2	Interactive single-property map	DONE — found already built, was mislabeled as not-started
3	Places Autocomplete on forms	DONE this session — migrated old Autocomplete class to PlaceAutocompleteElement in LocationPicker.tsx
4	Multi-property map view (browse page)	DONE this session, live-tested — split-screen list+map on /properties, hover to pin highlight, click-pin to scroll-to-card, filters sync both sides, mobile bug found and fixed
5	"Properties Near Me" radius search	DONE this session, live-tested — new sidebar control on /properties, pure client-side distance filter, zero new backend calls
6	Nearby Search + location-advantage badges	DONE this session — badges on cards and detail hero, straight-line distance from already-cached data, zero new API calls. Real limitation: a listing only shows a badge after its detail page has been viewed once (that's what populates the cache) — never fabricated
7	Routes API / Get Directions	DONE — found already fully built, no work needed
8	Street View toggle	CODE DONE, verified correct — BLOCKED on Google Cloud Console API key (see Blocker A, Section 5)
9	Natural-language smart search (e.g. "2BHK under 90L within 5km of Gachibowli")	NOT STARTED — deliberately deferred, needs its own dedicated design session, likely requires LLM-based query parsing

Separately, google.maps.Marker to AdvancedMarkerElement migration is blocked on a new Map ID + Cloud Console style profile — see Blocker B, Section 5.

7. Database, Feature & Admin Reference

Condensed from the full NILAY360_BLUEPRINT.md (attach that file in full alongside this document for complete detail — every claim there was verified against the live database, not assumed).

7.1 User roles (live user_role enum)

buyer, seller, agent, agency, moderator, content_manager, support, finance_manager, sales_manager, admin, super_admin, builder. Four roles matter day-to-day: Buyer/renter, Owner/seller, Agent (has agent_profiles row, public profile, large internal dashboard), Admin/super_admin (single unified /admin panel).

7.2 Core live tables (the real, active schema)

- property_listings — the real, live, seller-submitted listing table. Status values: pending_review, active, rejected, changes_requested, frozen
- agent_profiles — canonical agent record (status: pending/approved/rejected)
- inquiries — the real, live lead-management table (no CREATE TABLE migration exists for it; created out-of-band)
- deals — sale pipeline (stage: negotiation -> booking -> agreement -> registration -> closed / lost)
- documents, calendar_events, conversations/messages, deal_collaborators, tasks, teams/team_members, call_logs, legal_documents, consent_events, otp_requests, capture_360_requests, nearby_places_cache, property_view_events/property_image_clicks, listing_status_history, video_upload_sessions, change_log, site_contacts (new this session), admin_audit_log, reports, site_content
- Views: public_property_save_counts, public_agent_contact (narrow public column subset, protects sensitive profile fields)

7.3 Dead / superseded tables (confirmed zero references)

leads, lead_activities, agents (superseded by agent_profiles), agent_enquiries, agent_reviews, cities, neighbourhoods, developers, agencies, property_price_history, compare_sessions, property_views, cms_pages, banners, support_tickets, ticket_messages, audit_logs, feature_flags, seo_redirects, newsletter_subscribers, contact_submissions, appointments, reviews (generic), subscriptions, transactions, featured_listings.

Still-live-but-vestigial: the properties table (legacy seed/catalog data) is merged into 9 public pages as a fallback layer. It currently has 0 rows so it's a dormant risk, not an active bug — /locations and /compare were fixed this session to stop using it; /properties was NOT yet fixed (see reminder list, Section 8).

7.4 Admin panel — current sections

- Overview (stat cards + trend charts), Listings (unified, status-filterable, consolidated from 3 separate sections in an earlier session), Agents, 360 Requests, All Users, All Inquiries, Agent Performance, Leaderboard, Reports (the only path to freeze/unfreeze a listing), Site Content, Contact Numbers (new this session), Audit Log (read-only, permanent record of every admin write action)

7.5 Known technical debt / gaps (carried forward, unresolved)

- 4 migrations (035, 042, 045, 056) change only RLS policies with no accompanying schema object — cannot be verified via REST probe, need a direct pg_policies SQL check, never actually run
- A live RLS policy update_admin_all on property_listings exists in the real database but appears in no migration file — drift in the 'live but undocumented' direction
- conversations table's own RLS SELECT policy has a known, deliberately-left-alone recursion-shaped risk from earlier hardening work
- Two competing saved_properties table definitions exist across two different migrations — unclear which one actually won live
- Two inconsistent profile editors on the same profiles table: /profile/edit (4 fields) vs. Dashboard Profile tab (12+ fields) — should merge
- Sign-in results gate is active on /search, /buy, /rent, and the homepage, but NOT on /properties or /commercial — an inconsistency, needs one deliberate decision
- README.md is stale — claims Next.js 14, fake Algolia search, fake Razorpay/Stripe payments, fake Cloudflare hosting; needs a full rewrite

8. Complete Reminder List — Not Yet Built or Fixed

1. /properties — remove the dormant legacy properties table merge (0 rows today, but the same fabricated-data risk pattern already fixed on /locations and /compare)
2. /search — remove or repoint its old fake "Map placeholder" stub, now stale next to the real map view on /properties
3. Assign Agent dropdown — build the full advanced rebuild: real search/filter, pagination or virtual scroll for hundreds/thousands of agents, newest/oldest sort (only diagnosed so far, not built)
4. Set new password screen — the second half of the Forgot Password flow that's still missing
5. Watermarking — still completely unscoped: text vs. logo, placement, opacity, retroactive vs. new-uploads-only were never decided
6. Step-indicator wizard navigation — allow click-to-jump between steps in the post-property creation wizard
7. Router / ISP DNS issue — see Blocker D, Section 5
8. 4 unconfirmed RLS policy migrations — need a direct pg_policies SQL query, never actually run (see Section 7.5)
9. Admin download feature (bulk photos/videos per listing) — scoping questions never answered: ZIP vs. individual files, real Bunny source video vs. playback link, and the actual business reason it's needed
10. Homepage "Browse Properties" section directly after the search box — not started
11. Google Maps Phase 9 (natural-language smart search) — deferred, needs its own session
12. Video count per listing: 1 to up to 5 — scoped (cap confirmed at 5, needs its own table property_listing_videos mirroring the existing property_floor_plans pattern, plus a migration to preserve existing single-video listings) but not built
13. Two competing saved_properties table definitions — needs a live check to determine which one actually won
14. conversations RLS SELECT policy — known recursion-shaped risk, worth a dedicated look before heavy messaging use
15. README.md — full rewrite needed to match reality
16. Two competing profile editors — merge into one
17. Sign-in results gate inconsistency — needs one deliberate decision
18. Mobile web view (the old Capacitor wrapper) — confirmed poor UX across all pages; being superseded by the new native app rather than fixed directly (see Section 10)

9. Existing Web App Screen Inventory (condensed)

Condensed from NILAY360_APP_DESIGN_BRIEF.md — attach that file in full alongside this document for the complete screen-by-screen content/flow spec (~60 screens, every real action and state documented). This condensed version exists so the native app team has the full picture without needing to cross-reference immediately.

9.1 Stub screens — do NOT treat these as functional references

1. Forgot Password — now real (sends an actual email), but the 'set new password' screen after it is still missing (see Section 4.4 / 8)
2. Dashboard Settings tab — every Save/Update/Delete action is fake, sets local state only, nothing persists
3. Property Valuation (/valuation) — static 'Coming Soon' page, no real calculator
4. Public /saved route — static placeholder always showing empty; the REAL implementation is /dashboard/saved, use that as reference instead
5. Agent public profile 'Quick Contact' blur — a pure visual teaser, 'Unlock with NILAY360 Pro — coming soon,' no real payment/unlock flow exists
6. New Project Detail (/new-projects/[slug]) — shows the same generic placeholder regardless of which project was tapped, no real per-project data fetch
7. Agent Partner Landing (/agent-register) — pure marketing copy; the real application form is a separate screen, /become-an-agent
8. 'Connect' link hub (/connect) — the page itself is real, but several tiles are explicit placeholders (booking, support tickets, resource downloads, app-store badges, newsletter signup all point to WhatsApp or do nothing)

9.2 Real but inconsistent — decide ONE behavior for the native app

- Sign-in results gate: active on /search, /buy, /rent, homepage; NOT active on /properties, /commercial — pick one consistent rule for native
- Login/Register: real standalone pages exist, but production shows them as an overlay modal over the homepage instead — native app will presumably use a dedicated screen regardless
- Two profile editors (4 fields vs. 12+ fields) — use the fuller field set as the native reference, not both

9.3 Fully real, fully wired — safe to treat as reference (by area)

- Property Discovery: Homepage, Browse Grid, Search Results, Property Detail (full, with owner/agent self-view distinction), Buy, Rent, Commercial, New Projects (list), Locations (index + detail), Compare (up to 3 properties), Saved Properties (dashboard version)
- Listing Creation & Editing: 8-step creation wizard (Type, Location, Details, Pricing/Brokerage, Amenities, Photos/Video, Floor Plans/360 request, Review/Submit), single-page Edit form, My Listings management screen with real per-listing stats and 360-capture status

- Agent-Facing: Public agent profile, public agent directory, Leads (list + detail), Deals (pipeline, list + detail), Site Visits (list + detail), Messages (list + thread), Tasks (list + detail), Teams (list + detail with live aggregate dashboard), Calendar (unified: own events + assigned visits + lead follow-ups), Analytics, Leaderboard, Become an Agent (the real application form)
- Buyer Account & Utility: Login, Register, Auth Error page, Main Dashboard (Overview/My Listings/Saved/Recent Searches/Profile/My Inquiries/Appointments/Settings tabs), Saved Searches, Notifications (full page + navbar bell, 6 real trigger types plus a 7th cron-driven source), Profile Edit, Calculator Suite (5 genuinely functional tools: EMI, Affordability, Stamp Duty by state, Rent vs Buy, Loan Eligibility — all pure client-side, zero backend), Investment Calculator (redirects into the Affordability tab), Contact page (real inquiry form + office locations), Site-Visit Request modal (works for both signed-in and anonymous visitors)
- Admin Panel: Overview, Listings (unified moderation queue), Listing Preview modal, Agents, 360 Requests, All Users, All Inquiries, Agent Performance, Leaderboard, Reports (moderation/grievance queue with SLA tracking), Site Content, Contact Numbers, Audit Log

10. Native Mobile App — Product & Technical Requirements Document

STANDING DECISION, CONFIRMED: The native app is being built fresh in REPLIT. It is NOT the existing Capacitor wrapper (built by Vanith's brother) — that wrapper is being superseded, not extended. This is a ground-up native mobile frontend.

10.1 Product Vision

Build the single best, most premium real estate app experience available to Indian users — matching or exceeding the polish, speed, and visual quality of category leaders (Zillow, 99acres, NoBroker, MagicBricks) while carrying NILAY360's own premium brand identity (Deep Navy, Champagne Gold, Manrope/Inter typography, calm and trustworthy visual language). The app should feel like the natural mobile-native evolution of the NILAY360 brand — not a generic real-estate template, and not a thin wrapper around the website.

Target platforms: iOS and Android, both from a single Replit-built codebase.

10.2 Non-Negotiable Architectural Constraint

The backend, database, authentication, storage, and all business logic MUST remain 100% identical to the existing web platform. This is not optional and should never be silently violated by any build agent:

- SAME Supabase project (xjdarhbzrpybqyqeeshy) — same tables, same RLS policies, same auth system, same storage buckets
- SAME user accounts — a user who registers on the website must be able to log into the app with the same credentials, and vice versa
- SAME property data — a listing created on the website appears identically in the app, and a listing created in the app (if the app supports creation) appears identically on the website
- NO duplicate backend, NO duplicate database, NO second Supabase project, NO parallel API layer that diverges from what the website already uses
- The app calls Supabase directly via its official client SDK (or through the same API routes the website already uses where server-side logic is required, e.g. video upload signing, OTP sending) — it does not reinvent these
- If a genuinely new backend capability is needed that doesn't exist yet, it should be added as a new, backward-compatible addition to the SAME backend (e.g. a new Supabase Edge Function, a new API route in the same Next.js project) — never as a separate system

10.3 Recommended Technology

React Native + Expo is the recommended foundation, for these reasons specific to this project:

- Single codebase covers both iOS and Android
- Excellent, well-documented Supabase JS client compatibility (same @supabase/supabase-js patterns the web team already knows)

- Strong camera/media library support — needed for property photo/video capture and upload flows
- Mature push notification support (Expo Notifications) for the real-time-feeling experience a premium app needs (new lead alerts, message received, listing approved, etc.)
- Genuinely production-ready for App Store / Play Store deployment, not a prototyping-only tool
- Replit has strong, current support for scaffolding and iterating on Expo/React Native projects

Confirm this technology choice is actually what Replit's agent proposes once it has inspected the existing codebase — do not force it blindly if Replit's own investigation surfaces a strong reason to deviate. But this is the informed starting recommendation.

10.4 NEW Feature — Reels / Video Feed Section

A genuinely new feature, not present anywhere on the current website, requested explicitly for the native app: a short-form vertical video feed ("Reels"), similar in interaction pattern to Instagram Reels or YouTube Shorts, but built around real property content.

10.4.1 Content source

- Primary source: the property walkthrough videos already uploaded via Bunny Stream as part of the existing 8-step listing wizard's Photos & Video step. These already exist in the database (video_asset_provider, video_asset_id, video_asset_status, video_asset_thumbnail_url on property_listings) — the Reels feed should be built by querying and surfacing THIS existing data, not a new, separate video-upload system.
- Once the 1-to-5-video-per-listing upgrade (reminder item, Section 8) is built, the Reels feed naturally benefits — more source content per listing, no additional Reels-specific engineering needed for that part.
- Consider (as a future phase, not required for v1): allowing agents to upload Reels-specific short vertical clips distinct from the main listing walkthrough video, for more editorial/promotional content. Flag this as a decision point, not a requirement, for the initial build.

10.4.2 Interaction model

- Full-screen, vertical, swipe-to-advance video feed, auto-playing, muted-by-default with a tap-to-unmute affordance — the standard, expected pattern for this content type
- Each video overlays: property price, locality/city, a save/heart action, a share action, and a tap-through to the full Property Detail screen
- Feed ordering: start with a sensible default (e.g. newest-listed-first, or featured/premium-first) — this is a product decision Vanith should confirm before build, not something to leave to the build agent's default guess

10.4.3 Data & backend requirements

- No new video storage system — reuses Bunny Stream exactly as-is

- Likely needs one new, lightweight query pattern (e.g. `property_listings` filtered to `status=active` AND `video_asset_status=ready`, ordered appropriately) — this is a read-only addition, not a schema change, and should not require a new table unless engagement metrics (view count, like count per Reel) are wanted, in which case a new, clearly-scoped table (e.g. `reel_engagement_events`) should be proposed and reviewed before being built, not silently added

10.5 Core Screen Set for v1

Build against the REAL screen inventory in Section 9.3 of this document (and the full `NILAY360_APP_DESIGN_BRIEF.md` attached alongside it) as the source of truth for what each screen actually contains and does. Do not design from assumption.

- Onboarding / Auth: Login, Register (buyer/agent/builder account types), the real password-reset flow (once the missing 'set new password' screen is also built — this is a good moment to close that gap for both platforms at once)
- Discovery: Home (including the new Reels entry point), Browse/Search with real filters, Property Detail (full), Saved Properties, Compare
- NEW: Reels feed (Section 10.4)
- Listing (for owners/agents): a mobile-adapted creation/edit flow reusing the same 8-step field structure as the web wizard — do not invent a different field set
- Agent workspace: Leads, Deals, Site Visits, Messages, Tasks, Calendar, Analytics — these are real, data-dense screens; prioritize a genuinely usable mobile information architecture over cramming the web layout onto a small screen
- Account: Dashboard (Overview/My Listings/Saved/Profile/My Inquiries/Appointments), Notifications (push-notification-integrated, not just in-app), Settings (build this one for REAL from day one on native — do not replicate the website's fake Settings tab)

10.6 Explicit Stub-Handling Instructions for the Build Agent

- Forgot Password: build as fully real end-to-end, including the missing 'set new password' screen (close this gap for the FIRST time here, on native, then backport the fix to web)
- Property Valuation, Agent contact-info unlock: design as honest 'Coming Soon' states, never a fake working button
- New Project Detail: either fetch and display real per-project data, or show a simple, honest 'Contact us for details' card — never the same fake placeholder regardless of which project was tapped
- Skip entirely (not real features): public /saved (use /dashboard/saved's real version instead), Agent Partner Landing marketing page (use /become-an-agent's real form instead)
- 'Connect' hub equivalent: build only the real parts (WhatsApp/Call/Email via the new `site_contacts` system from Section 4.1) as a clean 'Get in Touch' screen — drop every fake placeholder tile (booking, support tickets, downloads, app badges, newsletter)

10.7 Security & Data Requirements (non-negotiable)

- Never bundle server-side secrets (SUPABASE_SERVICE_ROLE_KEY, MSG91_AUTH_KEY, CLOUDINARY_API_SECRET, etc.) into the mobile app bundle — only NEXT_PUBLIC_-equivalent, genuinely client-safe values belong in the app
- Respect every existing RLS policy exactly as-is — the mobile app is just another authenticated client hitting the same Supabase project, not a privileged one
- Session/token handling must follow Supabase's own recommended mobile SDK patterns (secure storage, not plain AsyncStorage, for auth tokens)
- Push notification tokens and deep links must be handled securely — no sensitive data embedded in a deep link URL
- Camera/media permissions requested only when actually needed (e.g. when a user starts the listing-creation flow), with clear purpose strings

10.8 Testing Requirements Before Any Release

- A property created on the website must appear correctly in the app, and vice versa (if app-side creation is in scope for v1)
- A user account created on the website must log into the app, and vice versa
- A profile/listing update made in one client must reflect correctly in the other
- Push notifications must reach the correct user for the correct event, matching the same 6 (soon 7, once WhatsApp Business is fixed) trigger types already defined on web
- Role-based permissions (buyer/seller/agent/admin) must behave identically to web — no accidental privilege differences introduced by a different client
- Do not assume synchronization works just because both the website and the app compile and run — test the actual round-trip.

10.9 Development Phases (recommended sequencing)

1. Repository & backend audit (Replit's agent inspects the actual Supabase schema, RLS, existing API routes — does not assume)
2. Mobile architecture proposal + technology confirmation (React Native + Expo, pending Replit's own investigation)
3. Project setup + shared Supabase client wiring
4. Authentication (login, register, session handling — proven working against real accounts before anything else)
5. Home + property discovery (browse, search, detail)
6. Reels feed (Section 10.4)
7. Saved properties + Compare
8. Listing creation/edit (owner/agent)
9. Agent workspace screens (Leads, Deals, Site Visits, Messages, Tasks, Calendar)

- 10.** Notifications (in-app + push)
- 11.** Settings (built real, not fake)
- 12.** Full round-trip testing against the live web platform (Section 10.8)
- 13.** Android build + internal testing
- 14.** iOS build + internal testing
- 15.** Production deployment (Play Store + App Store submission)

Do not jump ahead to build steps. Each phase should be reviewed before moving to the next, same working discipline already established on the web side of this project.

11. Master Prompts — Ready to Send

11.1 Master Prompt — Web / Claude Code Continuation

Use this to resume WEB platform development in a new Claude conversation.

This is a continuation of an ongoing NILAY360 development project. A complete handoff document (this PDF) has been provided as full context. Do NOT rebuild, do NOT assume architecture, do NOT guess – inspect the actual repository, database, and deployment directly before any new work.

Before anything else, confirm current live state matches what's documented:

1. git log/git status – confirm local HEAD still matches origin/main at commit c41c6d3, and that the 19 modified + 6 untracked files listed in Section 3 of this document are still present, uncommitted, unpushed.
2. Confirm migration 075 (site_contacts) is still applied live in Supabase.
3. Confirm whether the 4 active blockers in Section 5 have been resolved since this document was generated – ask directly rather than assuming either state.
4. Confirm the WhatsApp env vars (MSG91_WHATSAPP_INTEGRATED_NUMBER, MSG91_WHATSAPP_TEMPLATE_NAME) status in Vercel production.

Report full current state before proceeding with any new feature work. Once confirmed, the next priority items are listed in Section 8 (Complete Reminder List) of this document – work through them in the order I specify, one at a time, with the same investigate-before-build discipline used throughout this project: confirm real current state, propose a plan, wait for approval, then build and verify (tsc + eslint + live test) before moving to the next item. Never commit or push to git without my explicit, direct go-ahead, every single time.

11.2 Master Prompt — Native App / Replit Build

Use this as the FIRST message to Replit's build agent, with this entire document (and NILAY360_BLUEPRINT.md + NILAY360_APP_DESIGN_BRIEF.md) attached as context.

I am building a new, premium native mobile app for NILAY360, an existing live real-estate platform (nilay360.com). A complete Product & Technical Requirements Document is attached (see Section 10 specifically, plus Sections 1-9 for full existing-platform context). Read it in full before writing any code.

CRITICAL, NON-NEGOTIABLE CONSTRAINT: this app must use the EXACT SAME Supabase backend, database, authentication, and storage as the existing website. Do not create a new Supabase project. Do not create a new backend. Do not invent a parallel API. Do not duplicate user accounts or property data. This is a new FRONTEND for an existing platform, not a new platform. I will provide the real Supabase project URL and anon key separately, through a secure channel, not pasted in plain chat.

This is NOT the existing Capacitor-wrapped app (built separately by a family member) – that project is being superseded. Build this fresh, ground-up, as a genuinely native mobile experience, ideally React Native + Expo (confirm this is still the right call after your own investigation of what's easiest to wire against the existing Supabase setup).

FIRST TASK, before any UI work: inspect and report back on

1. The real Supabase schema (tables, RLS policies) relevant to a mobile client - auth, property_listings, profiles, agent_profiles, inquiries, saved_properties, notifications, site_contacts.
2. What existing web API routes (if any) should be reused rather than reimplemented (e.g. video upload signing, OTP sending).
3. Your recommended Expo project structure and initial screen list, cross-referenced against Section 9 and Section 10.5 of the attached document.

Do not begin building screens until I've reviewed and approved this initial report. Once approved, build in the phase order specified in Section 10.9 - one phase at a time, tested and reviewed before moving to the next. The new Reels feed (Section 10.4) is a genuinely new feature - build it against the existing Bunny Stream video data already attached to real listings, do not invent a separate video system for it.

The visual language must match NILAY360's existing premium brand: Deep Navy (#07111F), Primary Blue (#0B1F3A), Champagne Gold (#C9A227), Manrope and Inter typography. This should feel like the best real-estate app available to Indian users - calm, premium, minimal, trustworthy, fast - never a generic real-estate template.

12. Final Steps Checklist

Do these yourself, outside any AI session

1. Fix the Street View API key in Google Cloud Console (Blocker A, Section 5)
2. Provision a Map ID + dark style profile for the Marker migration, whenever convenient (Blocker B, Section 5) — not urgent
3. Check MSG91's dashboard for the real WhatsApp template approval status and add the two missing Vercel env vars (Blocker C, Section 5)
4. Address the router/ISP DNS issue whenever convenient (Blocker D, Section 5) — not urgent, but keep it in mind before assuming a future connectivity scare is a real site bug
5. Decide the still-open product questions flagged in this document: watermarking scope, Reels feed ordering default, whether agents get a separate Reels-upload flow in a future phase, the sign-in-gate consistency decision for native

Do these to start the next AI sessions

1. For continuing WEB development: open a new Claude conversation, paste this entire document, then send the prompt in Section 11.1
2. For starting the NATIVE APP build: open Replit, paste this entire document plus the two attached blueprint/design files as project context, then send the prompt in Section 11.2 — provide the real Supabase URL/anon key through Replit's secrets/environment feature, never in plain chat text
3. Keep both sessions honest: require them to verify real, current state (git status, live database checks, live Cloud Console/MSG91 status) before trusting anything in this document as still accurate — this document is a snapshot, not a live feed

Keep this document alive

Regenerate a document like this after any major future session — it is the cheapest insurance against re-discovering something that already exists, or silently losing a decision that was already made. Three separate times this session alone, real work was found already-built simply by investigating first instead of assuming. Keep that habit.